SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 11.2

Program	**: B. Tech (CSE)**
Specialization	**: -**
Course Title	**: AI Assisted Coding**
Course Code	**: 23CS002PC304**
Semester	**II**
Academic Session	**: 2025-2026**
Name of Student	**: A.VamshiKrishna**
Enrollment No.	**: 2403A51L47**
Batch No.	**52**
Date	**: 13/03/26**

Submission Starts here

Screenshots:

Task Description #1 (Stack Using AI Guidance)
• Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations

prompt:
Design and implement a Stack data structure in Python.

Requirements:
1. Create a class named Stack.
2. Implement the following basic stack operations:
   - push(item): Add an element to the top of the stack.
   - pop(): Remove and return the top element of the stack.
   - peek(): Return the top element without removing it.
   - is_empty(): Check if the stack is empty.
   - size(): Return the number of elements in the stack.
3. Use a Python list internally to store stack elements.
4. Add proper comments and a docstring explaining the stack and its operations.
5. Include a simple test section that demonstrates push, pop, and peek operations.
6. Print the stack contents after each operation.

```python
# Task Description -1 – (Stack Using AI Guidance)
# • Task: With the help of AI, design and implement a Stack data
# structure supporting basic stack operations.
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.stack.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack. Raises an error if the stack is empty."""
        if not self.is_empty():
            return self.stack.pop()
        else:
            raise IndexError("Stack is empty")

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises an error if the stack is empty."""
        if not self.is_empty():
            return self.stack[-1]
        else:
            raise IndexError("Stack is empty")

    def is_empty(self):
        """Check if the stack is empty."""
        return len(self.stack) == 0

    def size(self):
        """Return the number of items in the stack."""
        return len(self.stack)
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
```

Output:

```python
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print("Top item:", stack.peek())  # Output: Top item: 3
    print("Stack size:", stack.size())  # Output: Stack size: 3
    print("Popped item:", stack.pop())  # Output: Popped item: 3
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/ai_coding.py
Top item: 3
Stack size: 3
Popped item: 3
Stack is empty: False
PS C:\Users\Eshwar\OneDrive\Desktop\python>
```

Explaination:

1. The prompt asks AI to create a **Stack data structure in Python** using a class.
2. It includes basic stack operations such as **push, pop, peek, is_empty, and size**.
3. It also requires **comments, documentation, and a test example** to demonstrate how the stack works.

Task Description #2 - (Queue Design)

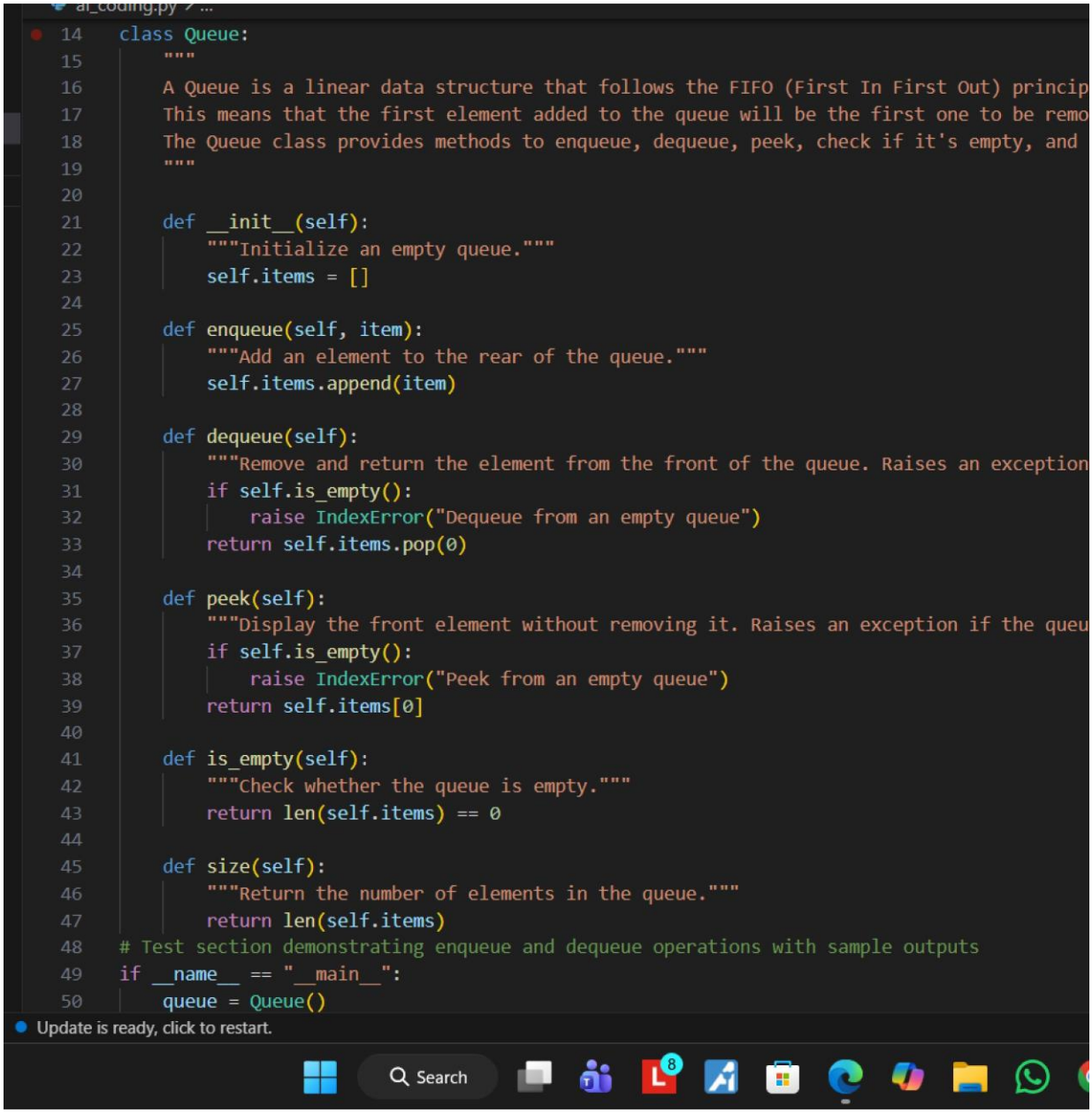
• Task:Use AI assistance to create a Queue data structure

following FIFO principles

Prompt:

Design and implement a Queue data structure in Python following the FIFO (First In First Out) principle.

Requirements:
1. Create a class named Queue.
2. Implement the following operations:
   - enqueue(item): Add an element to the rear of the queue.
   - dequeue(): Remove and return the element from the front of the queue.
   - peek(): Display the front element without removing it.
   - is_empty(): Check whether the queue is empty.
   - size(): Return the number of elements in the queue.
3. Use a Python list to store queue elements.
4. Add proper comments and a docstring explaining the Queue and FIFO concept.
5. Include a small test section demonstrating enqueue and dequeue operations with sample outputs.



```python
class Queue:
    """
    A Queue is a linear data structure that follows the FIFO (First In First Out) princip
    This means that the first element added to the queue will be the first one to be remo
    The Queue class provides methods to enqueue, dequeue, peek, check if it's empty, and
    """

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """Add an element to the rear of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the element from the front of the queue. Raises an exception
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
        return self.items.pop(0)

    def peek(self):
        """Display the front element without removing it. Raises an exception if the queu
        if self.is_empty():
            raise IndexError("Peek from an empty queue")
        return self.items[0]

    def is_empty(self):
        """Check whether the queue is empty."""
        return len(self.items) == 0

    def size(self):
        """Return the number of elements in the queue."""
        return len(self.items)
# Test section demonstrating enqueue and dequeue operations with sample outputs
if __name__ == "__main__":
    queue = Queue()
```

ai_coding.py > ...

```
51
52      # Enqueue elements
53      queue.enqueue(1)
54      queue.enqueue(2)
55      queue.enqueue(3)
56
57      print("Queue after enqueuing 1, 2, 3:", queue.items)  # Output: [1, 2, 3]
58
59      # Peek at the front element
60      print("Front element (peek):", queue.peek())  # Output: 1
61
62      # Dequeue elements
63      print("Dequeued element:", queue.dequeue())  # Output: 1
64      print("Queue after dequeuing:", queue.items)  # Output: [2, 3]
65
66      print("Is the queue empty?", queue.is_empty())  # Output: False
67      print("Size of the queue:", queue.size())  # Output: 2
68      # Dequeue remaining elements
69      print("Dequeued element:", queue.dequeue())  # Output: 2
70      print("Dequeued element:", queue.dequeue())  # Output: 3
71      print("Is the queue empty?", queue.is_empty())  # Output: True
72      # Attempting to dequeue from an empty queue will raise an exception
73      try:
74          queue.dequeue()
75      except IndexError as e:
76          print(e)  # Output: Dequeue from an empty queue
77      # Attempting to peek from an empty queue will raise an exception
78      try:
79          queue.peek()
80      except IndexError as e:
81          print(e)  # Output: Peek from an empty queue
82
83
```

Output:

```
PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/ai_codin
Queue after enqueuing 1, 2, 3: [1, 2, 3]
Front element (peek): 1
Dequeued element: 1
Queue after dequeuing: [2, 3]
Is the queue empty? False
Size of the queue: 2
Dequeued element: 2
Dequeued element: 3
Is the queue empty? True
Dequeue from an empty queue
Peek from an empty queue
PS C:\Users\Eshwar\OneDrive\Desktop\python>
```

Explaination:

The task asks AI to design and implement a Queue data structure in Python.
The queue must follow the FIFO (First In First Out) principle where the first element added is the first removed.
It should include basic operations like enqueue, dequeue, peek, is_empty, and size to manage queue elements.

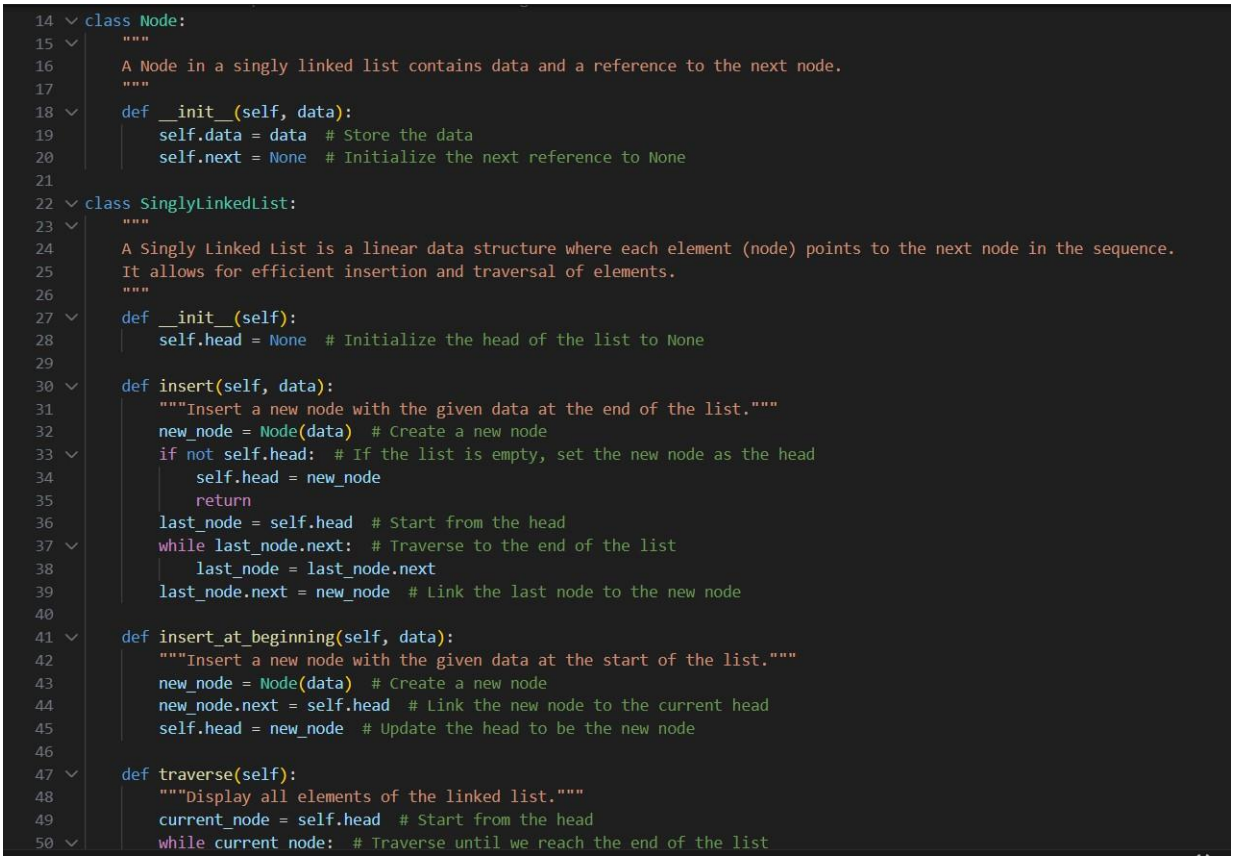
Task Description #3- (Singly Linked List Construction)
• Task: Utilize AI to build a singly linked list supporting insertion and traversal.

SR UNIVERSITY

Prompt:

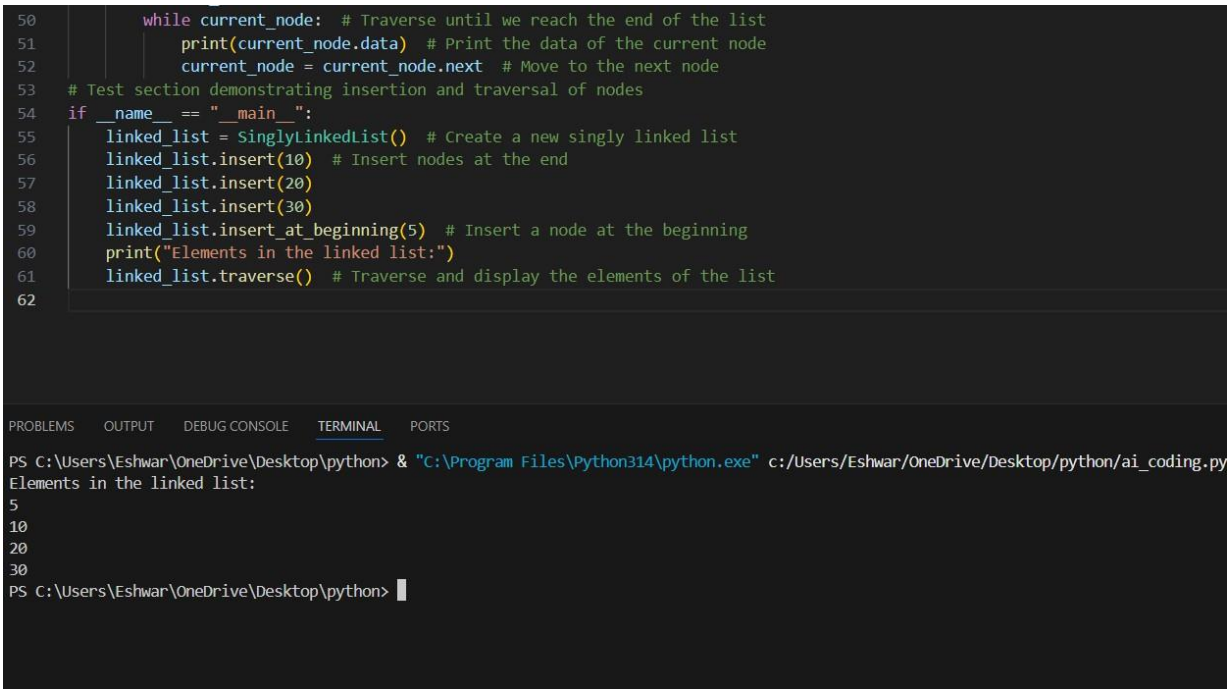
Design and implement a Singly Linked List in Python.

Requirements:
1. Create a Node class containing data and a reference to the next node.
2. Create a SinglyLinkedList class.
3. Implement the following operations:
   - insert(data): Insert a new node at the end of the list.
   - insert_at_beginning(data): Insert a node at the start of the list.
   - traverse(): Display all elements of the linked list.
4. Add comments and a docstring explaining how a singly linked list works.
5. Include a simple test section demonstrating insertion and traversal of nodes.



```python
class Node:
    """
    A Node in a singly linked list contains data and a reference to the next node.
    """
    def __init__(self, data):
        self.data = data  # Store the data
        self.next = None  # Initialize the next reference to None

class SinglyLinkedList:
    """
    A Singly Linked List is a linear data structure where each element (node) points to the next node in the sequence.
    It allows for efficient insertion and traversal of elements.
    """
    def __init__(self):
        self.head = None  # Initialize the head of the list to None

    def insert(self, data):
        """Insert a new node with the given data at the end of the list."""
        new_node = Node(data)  # Create a new node
        if not self.head:  # If the list is empty, set the new node as the head
            self.head = new_node
            return
        last_node = self.head  # Start from the head
        while last_node.next:  # Traverse to the end of the list
            last_node = last_node.next
        last_node.next = new_node  # Link the last node to the new node

    def insert_at_beginning(self, data):
        """Insert a new node with the given data at the start of the list."""
        new_node = Node(data)  # Create a new node
        new_node.next = self.head  # Link the new node to the current head
        self.head = new_node  # Update the head to be the new node

    def traverse(self):
        """Display all elements of the linked list."""
        current_node = self.head  # Start from the head
        while current_node:  # Traverse until we reach the end of the list
```

Output:



```
        while current_node:  # Traverse until we reach the end of the list
            print(current_node.data)  # Print the data of the current node
            current_node = current_node.next  # Move to the next node
# Test section demonstrating insertion and traversal of nodes
if __name__ == "__main__":
    linked_list = SinglyLinkedList()  # Create a new singly linked list
    linked_list.insert(10)  # Insert nodes at the end
    linked_list.insert(20)
    linked_list.insert(30)
    linked_list.insert_at_beginning(5)  # Insert a node at the beginning
    print("Elements in the linked list:")
    linked_list.traverse()  # Traverse and display the elements of the list

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/ai_coding.py
Elements in the linked list:
5
10
20
30
PS C:\Users\Eshwar\OneDrive\Desktop\python>
```

Explaination:

1. The prompt asks AI to **create a Singly Linked List in Python** using Node and LinkedList classes.
2. It should support **insertion of nodes and traversal** to display all elements.
3. The program should include **comments and example usage** to demonstrate how the linked list works.

Task Description -4 – (Binary Search Tree Operations)
• Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Prompt:

Design and implement a Binary Search Tree (BST) in Python.

Requirements:
1. Create a Node class containing data, left child, and right child.
2. Create a BinarySearchTree class.
3. Implement the following operations:
   - insert(data): Insert a new node into the BST following BST rules.
   - inorder_traversal(): Traverse the tree in inorder and display elements.
   - preorder_traversal(): Traverse the tree in preorder.
   - postorder_traversal(): Traverse the tree in postorder.
4. Add comments and a docstring explaining how a Binary Search Tree works.
5. Include a test section demonstrating insertion of multiple values and displaying the traversal outputs.

ai_coding.py > Node

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
class BinarySearchTree:

    def __init__(self):
        self.root = None

    def insert(self, data):
        """Insert a new node with the given data into the BST."""
        if self.root is None:
            self.root = Node(data)
        else:
            self._insert_recursive(self.root, data)

    def _insert_recursive(self, current_node, data):
        if data < current_node.data:
            if current_node.left is None:
                current_node.left = Node(data)
            else:
                self._insert_recursive(current_node.left, data)
        else:
            if current_node.right is None:
                current_node.right = Node(data)
            else:
                self._insert_recursive(current_node.right, data)

    def inorder_traversal(self):
        """Traverse the tree in inorder and return a list of elements."""
        return self._inorder_recursive(self.root)

    def _inorder_recursive(self, node):
        elements = []
        if node:
            elements += self._inorder_recursive(node.left)
```

```
            elements.append(node.data)
            elements += self._inorder_recursive(node.right)
        return elements

    def preorder_traversal(self):
        """Traverse the tree in preorder and return a list of elements."""
        return self._preorder_recursive(self.root)

    def _preorder_recursive(self, node):
        elements = []
        if node:
            elements.append(node.data)
            elements += self._preorder_recursive(node.left)
            elements += self._preorder_recursive(node.right)
        return elements

    def postorder_traversal(self):
        """Traverse the tree in postorder and return a list of elements."""
        return self._postorder_recursive(self.root)

    def _postorder_recursive(self, node):
        elements = []
        if node:
            elements += self._postorder_recursive(node.left)
            elements += self._postorder_recursive(node.right)
            elements.append(node.data)
        return elements
# Test section demonstrating insertion of multiple values and displaying the traversal outputs
if __name__ == "__main__":
    bst = BinarySearchTree()
    values_to_insert = [7, 3, 9, 1, 5, 8, 10]
    for value in values_to_insert:
        bst.insert(value)

    print("Inorder Traversal:", bst.inorder_traversal())
```

Output:

```
print("Inorder Traversal:", bst.inorder_traversal())
print("Preorder Traversal:", bst.preorder_traversal())
print("Postorder Traversal:", bst.postorder_traversal())
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/
Inorder Traversal: [1, 3, 5, 7, 8, 9, 10]
Preorder Traversal: [7, 3, 1, 5, 9, 8, 10]
Postorder Traversal: [1, 5, 3, 8, 10, 9, 7]
PS C:\Users\Eshwar\OneDrive\Desktop\python>
```

Explaination:

1. The prompt asks AI to create a Binary Search Tree (BST) in Python using Node and BinarySearchTree classes.

2. It focuses on inserting elements according to BST rules and performing tree traversals (inorder, preorder, postorder).
3. The program should include comments and example tests to show how the BST operations work.

Task Description -5 – (Hash Table Implementation)
• Task: Create a hash table using AI with collision handling

Prompt:

Design and implement a Hash Table in Python.

Requirements:
1. Create a HashTable class.
2. Implement a hash function to convert keys into index values.
3. Handle collisions using a method such as chaining (linked lists) or linear probing.
4. Implement the following operations:
   - insert(key, value): Add a key–value pair to the hash table.
   - search(key): Retrieve the value associated with the key.
   - delete(key): Remove a key–value pair from the table.
5. Add comments and a docstring explaining hashing and collision handling.
6. Include a test section demonstrating insertion, searching, and deletion operations.

ai_coding.py > ...

```python
class HashTable:
    def __init__(self, size=10):
        """Initialize the hash table with a specified size."""
        self.size = size
        self.table = [[] for _ in range(size)]  # Create a list of empty lists for chaining

    def hash_function(self, key):
        """Compute the hash value for a given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Add a key-value pair to the hash table."""
        index = self.hash_function(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key
                return
        # If the key does not exist, add it to the chain
        self.table[index].append((key, value))

    def search(self, key):
        """Retrieve the value associated with the key."""
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v  # Return the value if found
        return None  # Return None if the key is not found

    def delete(self, key):
        """Remove a key-value pair from the table."""
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair
                return True  # Return True if deletion was successful
```

Output:

```python
        return False  # Return False if the key was not found
# Test section demonstrating insertion, searching, and deletion operations
if __name__ == "__main__":
    # Create a hash table
    hash_table = HashTable()

    # Insert key-value pairs
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    # Search for values
    print(hash_table.search("name"))  # Output: Alice
    print(hash_table.search("age"))   # Output: 30
    print(hash_table.search("city"))  # Output: New York
    print(hash_table.search("country"))  # Output: None (not found)

    # Delete a key-value pair
    print(hash_table.delete("age"))  # Output: True (deletion successful)
    print(hash_table.search("age"))   # Output: None (not found after deletion)
    print(hash_table.delete("age"))  # Output: False (already deleted)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/ai_coding.py
Alice
30
New York
None
True
None
False
False
PS C:\Users\Eshwar\OneDrive\Desktop\python>
```

Expllaination:

1. The prompt asks AI to create a Hash Table in Python for storing key–value pairs.
2. It must include a hash function and collision handling method such as chaining or linear probing.
3. The program should support insert, search, and delete operations with comments and example tests.